

Manual Completo: Submódulos Git para Iniciantes

Versão: 1.0

Data: 14 de Agosto de 2026

Autor: Equipe de Desenvolvimento

Índice

1. O que são Submódulos Git?
 2. Por que Usar Submódulos?
 3. Submódulos Neste Projeto
 4. Como Adicionar Submódulos a um Projeto Novo
 5. Como Adicionar Submódulos a um Projeto Existente
 6. Gerenciando Submódulos
 7. Detalhes de Cada Submódulo
 8. Troubleshooting
 9. Dicas Avançadas
-

1. O que são Submódulos Git?

Explicação Simples

Imagine que você tem um projeto grande, como uma casa. Dentro dessa casa, você pode ter quartos que são, na verdade, outras casas menores. Essas casas menores têm seus próprios donos e podem ser atualizadas independentemente.

No mundo do Git, **submódulos** são repositórios Git dentro de outros repositórios Git. Eles permitem que você:

- Mantenha código compartilhado entre vários projetos
- Atualize o código compartilhado em todos os projetos de uma vez
- Mantenha cada repositório independente e versionado separadamente

Analogia do Dia a Dia

Pense em submódulos como “atalhos inteligentes” para outros repositórios. Quando você faz `git clone` de um projeto com submódulos, o Git cria as pastas, mas não baixa o conteúdo automaticamente. Você precisa dar um comando extra para “ativar” esses submódulos.

2. Por que Usar Submódulos?

Vantagens

Vantagem	Explicação
Reutilização	Use o mesmo código em vários projetos sem copiar e colar
Atualização Centralizada	Atualize o código em um lugar e propague para todos os projetos
Isolamento	Cada submódulo tem seu próprio histórico de versões
Organização	Mantém o projeto principal limpo e organizado

Quando Usar?

- Quando você tem código que é compartilhado entre vários projetos
- Quando você quer manter uma biblioteca de ferramentas atualizada
- Quando trabalha em equipe e diferentes pessoas mantêm diferentes partes do código

3. Submódulos Neste Projeto

O projeto “Fábrica de Comunicação” possui os seguintes submódulos:

3.1 .token-economy

O que é: Infraestrutura compartilhada de economia de tokens para projetos Claude Code.

Skills incluídas: - lean-ctx - Economia de contexto - headroom - Compressão de logs - caveman - Respostas telegráficas - rtk-memory - Registro de erros e padrões - pre-flight-check - Validação antes de commit/deploy - calcular-gastos-sessao - Cálculo de gastos por sessão - fable-method - Loop de resolução de problemas - fable-judge - Verificação adversarial - self-learning - Captura de golden paths

3.2 .code-review-graph

O que é: Banco de dados de grafo de conhecimento para revisão de código. Armazena informações sobre a estrutura do código e permite consultas inteligentes.

Comunidades: - scripts-caminho - scripts-carregar - scripts-chechar - scripts-config - scripts-estado - scripts-resultado - scripts-rodar

3.3 tooling/kit-fundacao-aidd

O que é: Kit de fundação com 5 decisões de engenharia de software reutilizáveis.

As 5 peças: 1. **Builder ≠ Critic** - Quem gera nunca é quem aprova 2. **Crítico determinístico** - Checagem de formato é script, nunca LLM 3. **Registro declarativo** - Tipo novo = 1 entrada num dicionário 4. **Nunca commitar vermelho** - Hook mecânico, não promessa em texto 5. **Postmortem que vira teste** - Toda linha de “Prevenção” nasce com um teste

3.4 .claude/skills/impeccable

O que é: Skill para design de interfaces frontend. Permite criar, redesenhar, auditar e polir interfaces web com alta qualidade.

Capacidades: - Design e redesenho de interfaces - Crítica e auditoria de UX - Animações e micro-interações - Otimização de performance - Acessibilidade - Responsividade

4. Como Adicionar Submódulos a um Projeto Novo

Passo a Passo Detalhado

4.1 Criar o Repositório Principal

```
# 1. Crie uma pasta para o projeto
mkdir meu-novo-projeto
cd meu-novo-projeto

# 2. Inicialize o Git
git init

# 3. Crie o repositório remoto (no GitHub, GitLab, etc.)
# e associe-o ao seu projeto local
git remote add origin https://github.com/seu-usuario/meu-novo-projeto.git
```

4.2 Adicionar o Submódulo .token-economy

```
# 1. Adicione o submódulo
git submodule add https://github.com/Heverton-web/token-economy-shared.
git .token-economy

# 2. Execute o script de configuração
# No macOS/Linux:
bash .token-economy/setup.sh

# No Windows:
powershell -ExecutionPolicy Bypass -File .token-economy\setup.ps1

# 3. Verifique se foi adicionado corretamente
git status
```

4.3 Adicionar o Submódulo tooling/kit-fundacao-aidd

```
# 1. Crie a pasta tooling se não existir
mkdir -p tooling

# 2. Adicione o submódulo
git submodule add https://github.com/Heverton-web/kit-fundacao-aidd.git tooling/
kit-fundacao-aidd

# 3. Execute o diagnóstico
python tooling/kit-fundacao-aidd/analisar-projeto.py .
```

```
# 4. Instale as peças (em modo dry-run primeiro)
python tooling/kit-fundacao-aidd/instalar.py . --peca todas
```

```
# 5. Revise o plano e aplique
python tooling/kit-fundacao-aidd/instalar.py . --peca todas --aplicar
```

4.4 Adicionar o Submódulo `.code-review-graph`

```
# 1. Adicione o submódulo (se disponível como repositório separado)
# git submodule add <url-do-repositorio> .code-review-graph
```

```
# 2. Para projetos que já possuem o grafo, copie a pasta manualmente
# ou baixe do repositório de origem
```

4.5 Configurar a Skill `impeccable`

```
# 1. A skill já está em .claude/skills/impeccable/
# 2. Para usar, invoque no Claude Code:
# /impeccable [comando] [alvo]
```

4.6 Commitar as Alterações

```
# 1. Adicione todas as alterações
git add .
```

```
# 2. Committe
git commit -m "feat: adiciona submódulos de economia de tokens e kit de fundação"
```

```
# 3. Envie para o repositório remoto
git push -u origin main
```

5. Como Adicionar Submódulos a um Projeto Existente

Passo a Passo Detalhado

5.1 Navegue até o Projeto

```
cd /caminho/para/seu/projeto/existente
```

5.2 Verifique o Estado Atual

```
# Veja se há alterações pendentes
git status
```

```
# Se houver, committe ou stashe-as primeiro
git stash
```

5.3 Adicione os Submódulos

Siga os mesmos passos da seção 4.2, 4.3 e 4.4.

5.4 Resolva Conflitos (se houver)

Se o projeto já tiver uma pasta com o mesmo nome do submódulo:

```
# 1. Renomeie a pasta existente
mv pasta-existente pasta-existente-backup

# 2. Adicione o submódulo
git submodule add <url> nome-da-pasta

# 3. Migre os arquivos necessários
cp -r pasta-existente-backup/* nome-da-pasta/

# 4. Remova o backup
rm -rf pasta-existente-backup
```

6. Gerenciando Submódulos

6.1 Atualizar Submódulos

```
# Atualizar todos os submódulos
git submodule update --remote --merge

# Atualizar um submódulo específico
cd .token-economy
git pull origin main
cd ..
git add .token-economy
git commit -m "chore: atualiza token-economy"
```

6.2 Clonar um Projeto com Submódulos

```
# Opção 1: Clone com submódulos
git clone --recurse-submodules https://github.com/seu-usuario/projeto.git

# Opção 2: Clone normal e depois inicialize submódulos
git clone https://github.com/seu-usuario/projeto.git
cd projeto
git submodule update --init --recursive
```

6.3 Remover um Submódulo

```
# 1. Desative o submódulo
git submodule deinit -f .token-economy

# 2. Remova do índice
git rm -f .token-economy

# 3. Limpe a pasta .git/modules
rm -rf .git/modules/.token-economy

# 4. Committe
git commit -m "chore: remove submódulo .token-economy"
```

6.4 Listar Submódulos

Liste todos os submódulos

```
git submodule status
```

Liste submódulos com detalhes

```
git submodule summary
```

7. Detalhes de Cada Submódulo

7.1 .token-economy

Instalação

Adicione o submódulo

```
git submodule add https://github.com/Heverton-web/token-economy-shared.
```

```
git .token-economy
```

Configure

```
bash .token-economy/setup.sh # macOS/Linux
```

ou

```
powershell -ExecutionPolicy Bypass -File .token-economy\setup.ps1 # Windows
```

Skills Disponíveis

Skill	Função	Quando Usar
lean-ctx	Economia de contexto	Quando o contexto está ficando grande
headroom	Compressão de logs	Quando logs são muito longos
caveman	Respostas telegráficas	Quando quer respostas curtas
rtk-memory	Registro de erros	Para não repetir erros
pre-flight-check	Validação antes de commit	Antes de commitar mudanças
calcular-gastos-sessao	Cálculo de gastos	Para saber quanto gastou
fable-method	Resolução de problemas	Para resolver problemas complexos
fable-judge	Verificação adversarial	Para verificar se algo funciona
self-learning	Captura de golden paths	Para documentar caminhos que funcionaram

Uso das Skills

No Claude Code, invoque as skills assim:

```
/lean-ctx
```

```
/headroom
```

```
/caveman
```

```
/rtk-memory
```

```
/pre-flight-check  
/calcular-gastos-sessao  
/fable-method  
/fable-judge  
/self-learning
```

7.2 .code-review-graph

O que é o Grafo de Conhecimento?

O grafo de conhecimento é um banco de dados que mapeia relações entre partes do código. Ele permite:

- Encontrar código relacionado rapidamente
- Entender dependências entre arquivos
- Fazer perguntas sobre a estrutura do projeto

Como Usar

```
# Para consultar o grafo, use os scripts em scripts/  
# Exemplo: scripts/consultar_grafo.py "onde está a função X"
```

Comunidades no Grafo

Cada comunidade representa um grupo de código relacionado:

- **scripts-caminho**: Scripts que lidam com caminhos de arquivos
- **scripts-carregar**: Scripts que carregam dados
- **scripts-chechar**: Scripts que fazem validações
- **scripts-config**: Scripts de configuração
- **scripts-estado**: Scripts que gerenciam estado
- **scripts-resultado**: Scripts que processam resultados
- **scripts-rodar**: Scripts que executam ações

7.3 tooling/kit-fundacao-aidd

As 5 Peças em Detalhe

Peça 1: Builder ≠ Critic

O que é: Separa quem cria de quem avalia.

Por que é importante: Se a mesma pessoa criar e avaliar, ela tende a ser mais flexível com seus próprios erros.

Como instalar:

```
# O instalador cria:  
# - agents/builder.md (agente que cria)  
# - agents/critic.md (agente que avalia)
```

Peça 2: Crítico Determinístico

O que é: Validações que são scripts, não LLMs.

Por que é importante: Scripts são consistentes e não “esquecem” regras.

Como instalar:

```
# 0 instalador cria:  
# - scripts/validar_template.py.template
```

Peça 3: Registro Declarativo

O que é: Tipos novos são adicionados em um dicionário, não com if/else espalhados.

Por que é importante: Facilita manutenção e evita erros.

Como instalar:

```
# 0 instalador cria:  
# - scripts/registro_declarativo_scaffold.py
```

Peça 4: Nunca Commitar Vermelho

O que é: Hook de pre-commit que bloqueia commits com erros.

Por que é importante: Previne que código com erros seja commitado.

Como instalar:

```
# 0 instalador cria/modifica:  
# - .git/hooks/pre-commit
```

Peça 5: Postmortem que Vira Teste

O que é: Toda lição aprendida vira um teste de regressão.

****Por que é importantes** Previne que o mesmo erro aconteça novamente.

Como instalar:

```
# 0 instalador cria:  
# - templates/POSTMORTEM.md  
# - scripts/postmortem_para_teste.py
```

7.4 .claude/skills/impeccable

Configuração

```
# A skill já está instalada em .claude/skills/impeccable/  
# Para usar, invoque no Claude Code:  
/impeccable [comando] [alvo]
```

Comandos Disponíveis

Comando	Categoria	Descrição
shape [feature]	Build	Planejar UX/UI antes de escrever código
init	Build	Capturar contexto do produto
document	Build	Gerar DESIGN.md a partir do código
extract [target]	Build	Extrair tokens e componentes
critique [target]	Evaluate	Revisão de UX com pontuação
audit [target]	Evaluate	Checagens técnicas (a11y, perf)
polish [target]	Refine	Passagem final de qualidade
bolder [target]	Refine	Amplificar designs seguros
quieter [target]	Refine	Suavizar designs agressivos
distill [target]	Refine	Simplificar para a essência
harden [target]	Refine	Tornar production-ready
onboard [target]	Refine	Design de primeiros fluxos
animate [target]	Enhance	Adicionar animações
colorize [target]	Enhance	Adicionar cor estratégica
typeset [target]	Enhance	Melhorar tipografia
layout [target]	Enhance	Corrigir espaçamento
delight [target]	Enhance	Adicionar personalidade
overdrive [target]	Enhance	Ultrapassar limites convencionais
clarify [target]	Fix	Melhorar UX copy
adapt [target]	Fix	Adaptar para diferentes dispositivos
optimize [target]	Fix	Diagnosticar e corrigir performance
live	Iterate	Modo de variante visual

Exemplos de Uso

```
# Criar uma nova landing page  
/impeccable shape minha-landing-page
```

```
# Auditar acessibilidade  
/impeccable audit minha-pagina
```

```
# Tornar o design mais ousado  
/impeccable bolder meu-componente
```

```
# Adicionar animações
/impeccable animate meu-slider
```

8. Troubleshooting

Problema 1: Submódulo não foi baixado

Sintoma: A pasta do submódulo está vazia.

Solução:

```
git submodule update --init --recursive
```

Problema 2: Conflito ao adicionar submódulo

Sintoma: Mensagem de erro ao executar `git submodule add`.

Solução:

```
# Verifique se já existe uma pasta com esse nome
ls -la
```

```
# Se existir, renomeie ou remova
mv pasta-existente pasta-existente-backup
```

```
# Tente adicionar novamente
git submodule add <url> nome-da-pasta
```

Problema 3: Submódulo está “sujo”

Sintoma: `git status` mostra alterações no submódulo.

Solução:

```
# Navegue até o submódulo
cd .token-economy
```

```
# Veja as alterações
git status
```

```
# Descarte as alterações (se não forem importantes)
git checkout .
```

```
# Ou faça commit delas
git add .
git commit -m "chore: atualizações no token-economy"
```

Problema 4: Erro ao executar scripts do kit-fundacao-aidd

Sintoma: Mensagem de erro “Python não encontrado” ou “permissão negada”.

Solução:

```
# Verifique se Python está instalado
python --version
```

```
# No Windows, talvez precise usar python3
python3 tooling/kit-fundacao-aidd/analisar-projeto.py .
```

```
# No Linux/macOS, pode precisar de permissão de execução
chmod +x tooling/kit-fundacao-aidd/*.py
```

Problema 5: Skill impeccable não é encontrada

Sintoma: Mensagem de erro “skill not found” ao usar /impeccable.

Solução:

```
# Verifique se a pasta existe
ls -la .claude/skills/impeccable/

# Se não existir, clone o repositório da skill
# ou baixe manualmente
```

9. Dicas Avançadas

9.1 Trabalhando em Equipe

- **Documente os submódulos:** Crie um README que explique quais submódulos são usados e por quê
- **Use versões fixas:** Em vez de sempre pegar a última versão, especifique uma tag ou commit
- **Comunique atualizações:** Quando atualizar um submódulo, informe a equipe

9.2 Automação com CI/CD

```
# Exemplo de pipeline GitHub Actions
name: CI
```

```
on: [push, pull_request]
```

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          submodules: recursive

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
```

```
- name: Run tests
  run: |
    pytest
```

9.3 Boas Práticas

1. **Não modifique submódulos diretamente:** Faça alterações no repositório original
2. **Mantenha submódulos atualizados:** Regularmente execute `git submodule update --remote`
3. **Use .gitmodules:** Mantenha o arquivo `.gitmodules` versionado
4. **Documente dependências:** Liste quais submódulos são necessários no README
5. **Teste após atualizações:** Sempre teste o projeto após atualizar submódulos

9.4 Comandos Úteis para o Dia a Dia

```
# Ver status de todos os submódulos
git submodule status
```

```
# Atualizar todos os submódulos para a última versão
git submodule update --remote --merge
```

```
# Ver diferenças em um submódulo
git diff --submodule
```

```
# Inicializar submódulos após clone
git submodule update --init --recursive
```

```
# Ver log de um submódulo
cd .token-economy
git log --oneline -10
```

Conclusão

Submódulos Git são uma ferramenta poderosa para gerenciar código compartilhado entre projetos. Este manual cobriu:

1. O conceito de submódulos e por que usá-los
2. Como adicionar submódulos a projetos novos e existentes
3. Como gerenciar submódulos no dia a dia
4. Detalhes de cada submódulo disponível neste projeto
5. Soluções para problemas comuns
6. Dicas avançadas para uso profissional

Lembre-se: a chave para trabalhar com submódulos é **prática**. Comece com projetos simples e vá aumentando a complexidade à medida que se sentir confortável.

Suporte:

Se tiver dúvidas, consulte a documentação oficial do Git sobre submódulos:

<https://git-scm.com/book/br/v2/Git-Tools-Submodules>

ou entre em contato com a equipe de desenvolvimento.